

DNA 컴퓨팅을 이용한 언어처리
Language Processing
Using DNA Computing

지도교수 : 장병탁

이 논문을 공학학사 학위 논문으로 제출함.

2003년 6월 26일

서울대학교 공과대학
컴퓨터공학부
김홍석

2003년 8월

초 록

DNA 컴퓨팅은 기존의 실리콘기반 컴퓨터와는 전혀 다른 새로운 패러다임의 계산방법이다. 지금까지 존재해왔던 컴퓨팅 기법 혹은 방법론은 빠른 속도의 발전을 보여왔음에도 불구하고 이 것으로는 여전히 제대로 해결하지 못하는 다양한 문제들이 존재했다. DNA 컴퓨팅 기법은 이러한 문제들의 해결을 위한 무궁무진한 가능성을 지니고 있다.

본 논문에서는 DNA 컴퓨팅의 기초적인 연산자와 실험기법을 사용하여 특히 인간의 두뇌 속에서 이루어지는 단어와 관련된 언어처리 기법을 DNA 컴퓨팅 기법을 사용하여 모델링한다. word game이라고 불리는 단어 연상과정을 DNA 컴퓨팅을 이용하여 설계하고, 시뮬레이션을 통해 검증하였다. 또한 이 과정에서 파생하는 다양한 문제들을 해결하기 위한 기법을 밝혔다.

이러한 과정을 통해서 기존의 순차적이고 직렬적인 방법으로는 제대로 구현하기 어려웠던 초병렬적인 인간의 두뇌활동을 DNA 컴퓨팅으로 구현해볼 수 있는 가능성을 발견했다. 덧붙여 다양한 분야의 응용 가능성을 타진해보았다.

<제 목 차 례>

1. 서론	1
1.1. 연구의 배경	1
1.2. 연구의 목적	1
1.3. 논문의 구성	1
2. DNA 컴퓨팅	3
2.1. DNA 컴퓨팅의 기본개념	3
2.2. DNA 컴퓨팅의 세부실험과정	3
2.2.1. hybridization	3
2.2.2. melting	4
2.2.3. ligation	4
2.2.4. PCR	5
2.2.5. 겔 전기 영동법	5
2.3. DNA 컴퓨팅의 특징	6
3. word game	7
3.1. word game의 모델	7
3.2. 기존 시스템의 한계	7
3.3. 새로운 접근방법	8
4. 핵심 디자인	10
4.1. 문제점 및 해결 전략	10
4.1.1. 각 워드의 DNA 코드 디자인	10
4.1.2. query 디자인	10
4.1.3. output의 문제	11
4.1.4. 제한 및 가정	11
4.2. 실험 알고리즘	12
4.2.1. step1-query 생성	12
4.2.2. step2-query 투입	12
4.2.3. step3-hybridization	12
4.2.4. step4-gel electrophoresis	13
4.2.5. step5-melting	14
4.2.6. step6-gel electrophoresis	14
4.2.7. step7-sampling	14
4.2.8. step8-restore and goto step1	14
5. 시뮬레이션 및 결과	15
5.1. 실험내용	15

5.2. 실험방법	17
5.3. 실험결과	18
5.4. 결과분석	20
6. 결론	21
6.1. 연구 결과 요약	21
6.2. 연구 결과 활용 방안	21
6.3. 향후 과제	21
참고문헌	23
부록	24

<표 차례>

표 1 순차적이고 직렬적인 처리를 위한 테이블의 예	7
표 2 N의 크기에 따른 필요한 기억용량의 예	8
표 3 각 필드에 대한 특성값 지정	15
표 4 실험에 사용된 단어의 DNA 코드	15
표 5 실험에 관련된 핵심 조건 가정	18

<그림 차례>

그림 1 DNA 모델	3
그림 2 hybridization의 예	4
그림 3 melting의 예	4
그림 4 ligation의 예	4
그림 5 PCR의 세부단계	5
그림 6 겔 전기 영동법	5
그림 7 다루는 문제의 개략도,	7
그림 8 DNA 컴퓨팅을 이용한 word game 개략도	9
그림 9 query생성	12
그림 10 query투입	12
그림 11 word와 query의 hybridization	13
그림 12 겔 전기영동법을 통한 dsDNA분리	13
그림 13 겔 전기 영동법을 이용한 query와 word의 분리	14
그림 14 실험 word의 DNA 코드 구조	15
그림 15 TCP를 query로 투입했을 때 각 단어별 hybridization이 일어나는 빈도 측정값의 평균 그래프	19
그림 16 transaction과 Java를 query로 투입했을 때 각 단어별 hybridization이 일어나는 빈도측정값의 평균 그래프	19
그림 17 query의 word수에 따른 각 단어별 hybridization의 비율 그래프(1~3개 의 word query)	20

1. 서론

1.1. 연구의 배경

인간은 오랜 역사를 통하여 인간을 닮은 기계를 꿈꿔왔다. 과학과 기술의 발전을 통하여, 인간의 여러 활동 중 물리적인 면에 있어서 기계는 매우 많은 발전을 하였다. 그러나 이에 반하여 인간과 비슷한 인식, 추론, 학습 등의 고차원적 사고 능력을 갖고 있는 기계의 착상과 개발은 훨씬 뒤늦었다. 1950년대와 60년대의 태동기를 거쳐 70-80년대를 거치며 절정기를 맞았던 인공지능 분야의 연구는 수많은 노력에도 불구하고 많은 좌절을 겪어왔다. 이는 기존의 폰노이만 컴퓨터를 기반으로 한 순차적이고 직렬적인 방식을 통한 시스템의 구현과 적용이 병렬적인 연산방식으로 동작하는 지능을 구현하는데 커다란 어려움을 겪고 있었음을 알 수 있다. 이에 문제의식을 느끼고 70년대 후반부터 시작된 생체분자를 이용한 연산 또한 큰 성과 없이 지지부진한 상태였다.

이러한 위기 속에서 1993년 Adleman이 Science에 발표한 논문으로 촉발된 DNA 컴퓨팅의 개념은 그동안 컴퓨터분야에서 풀기 어려웠던 문제로 널리 알려졌던 NP-hard 문제들을 비롯한 새로운 영역과, 그 응용 분야에 있어 지금까지와는 완전히 다른 새로운 가능성을 열어 주고 있다. 특히 인간의 뇌속에서 이루어지는 연산과정이 가진 초병렬성을 구현하기 힘들었던 면에서 DNA 컴퓨팅의 등장은 매우 중요하고 이 문제를 DNA컴퓨팅으로 접근함으로써 초병렬성을 구현하고 이를 통해 인공지능 분야의 연구에도 또 하나의 획기적인 돌파구가 될 수 있다고 하겠다.

1.2. 연구의 목적

본 논문은 기존의 실리콘 기반의 전자기 컴퓨팅과는 전혀 다른 분자 컴퓨팅의 대표적인 방법인 DNA 컴퓨팅을 응용해본다. 아직 DNA컴퓨팅은 연구 초기이므로 고차원의 문제를 처리하기는 어려우나 본 논문에서는 word game으로 불리는 인간의 두뇌 속에서 이루어지는 간단한 수준의 언어처리 과정을 모델링하는데 DNA 컴퓨팅을 응용해보고 이로써 기존의 방법으로 풀기 어려웠던 문제들을 풀어내고자 하는 DNA 컴퓨팅의 응용 가능성에 대해 알아보도록 한다. 또한 그와 더불어 실제 실험과정에서 일어나는 어려움을 해결할 세부적인 방안에도 대해서도 알아보도록 한다.

1.3. 논문의 구성

1장에서는 이 연구의 배경과, 목적 그리고 구성을 서술하고 있으며 2장에서는 본 연구의 방법론이 되는 DNA컴퓨팅에 대해서 실험에 응용되는 실제적인

연산자를 중심으로 설명한다. 3장에서는 우리가 다루는 문제인 word game에 대해서 설명하고 기존의 방법론의 한계를 다루며 DNA컴퓨팅의 필요성을 설명한다. 이어지는 4장에서는 DNA컴퓨팅을 응용한 핵심적인 디자인을 설명하여, 구체적인 실험 프로토콜과 알고리즘 그리고 구현한 DNA 코드를 소개하고 설명한다. 5장은 4장에서 설계한 디자인을 컴퓨터 시뮬레이션을 통해 구현하여 DNA컴퓨팅의 가능성과 기존의 방법론과의 차이점을 검증한다. 마지막 6장에서는 본 연구의 결과를 요약하고 미진한 부분과 문제점, 그리고 본 연구를 통한 향후의 활용방안에 대해 다룬다.

2. DNA 컴퓨팅

2.1. DNA 컴퓨팅의 기본개념

DNA컴퓨팅은 DNA를 이용해서 정보를 전달하고 처리한다. 따라서 DNA 컴퓨팅의 기본 정보단위는 DNA를 구성하고 있는 A, C, G, T의 네가지 염기이다. A와 T, 그리고 C와 G가 결합하게 되는데 이를 상보 결합이라고 부르고 이 결합과 분리를 통해서 기본적인 연산이 이루어진다. 이러한 결합은 매우 많은 수의 분자들 사이에서 동시에 병렬적으로 이루어지며, 이를 통해 DNA 컴퓨팅의 초병렬성이 보장된다. 또한 이러한 병렬성에 비하여 실질적으로 점유되는 공간과, 에너지는 매우 미미하다.

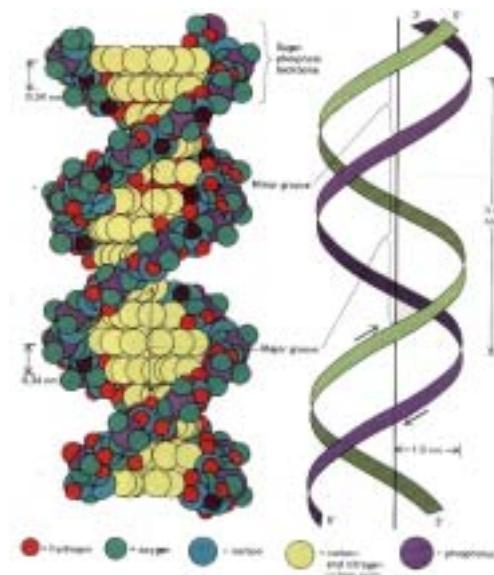


그림 1 DNA 모델

2.2. DNA 컴퓨팅의 세부실험과정

DNA 컴퓨팅을 실질적으로 실험하기 위해서는 DNA분자들을 조작하는 생물학적인 실험방법이 필요하다. Adleman이 처음 DNA컴퓨팅에 관한 논문을 발표했을 때만해도 초보적인 단계에 머물렀던 DNA관련 조작기술은 이제 상당한 수준에 이르렀다. 연산으로 사용되는 DNA분자를 조작하는데 관한 세부적인 실험과정은 매우 여러 가지이고 세세한 부분까지 살펴보자면 무척 복잡하나 본 논문에서는 이후의 실험설계에서 사용된 것을 중심으로 하여 간략히 살펴보도록 한다.

2.2.1. hybridization

상보적인 두 염기가 서로 수소결합을 통하여 결합하는 것이 염기서열인

DNA에서 일어나서 두 분자가 결합하는 것으로서 DNA컴퓨팅에서 가장 핵심적인 연산이다. ssDNA (single strand DNA)는 이 연산을 통해서 dsDNA(double strand DNA)가 된다.

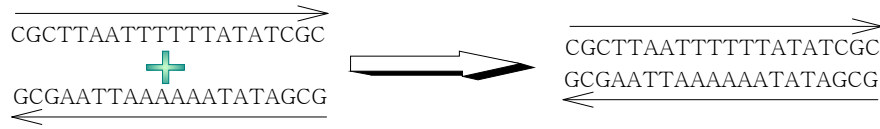


그림 2 hybridization의 예

2.2.2. melting

hybridization의 반대되는 개념으로서 dsDNA가 들어있는 시험관의 온도를 높여주면 두 DNA분자사이의 수소결합의 결합력이 약해지면서 두개의 ssDNA로 분리되는 연산이다. dsDNA의 염기 배열에 따라서 결합력이 차이가 나므로 온도를 잘 조절해주면 원하는 배열의 dsDNA만을 분리해낼 수 있 search기능의 역할도 할 수 있다.

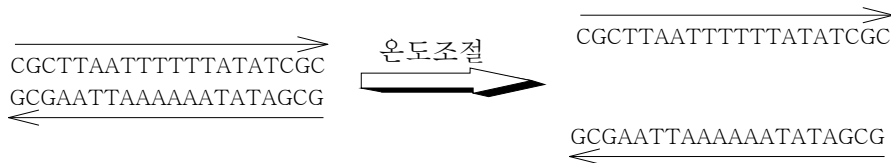


그림 3 melting의 예

2.2.3. ligation

두개의 ssDNA를 하나의 ssDNA로 연결하는 연산이다. 이는 hybridization과 melting과 함께 연동하여 사용할 수 있다. glue strands에 각각 hybridization된 두개의 ssDNA의 닿아있는 부분을 ligaze라는 효소를 이용하여 붙여주고 melting으로 glue strands를 떼어내면 두개의 ssDNA가 하나의 ssDNA로 이어진다.

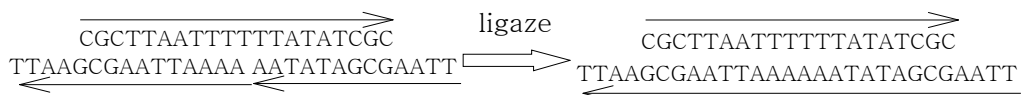


그림 4 ligation의 예

2.2.4. PCR

특정한 DNA를 복사해서 많은 수의 같은 DNA조각들을 만들어내는 방식이다. 이 동작을 위해서는 hybridzaion과 ligation 그리고 melting동작이 반복되어 사용되며 한번의 단계를 통해 이론상 2배로 증폭될 수 있다. 그러므로 n번 거치면 이를 통해서 2^n 개의 처음 DNA와 같은 배열의 염기가 생성될 수 있다.

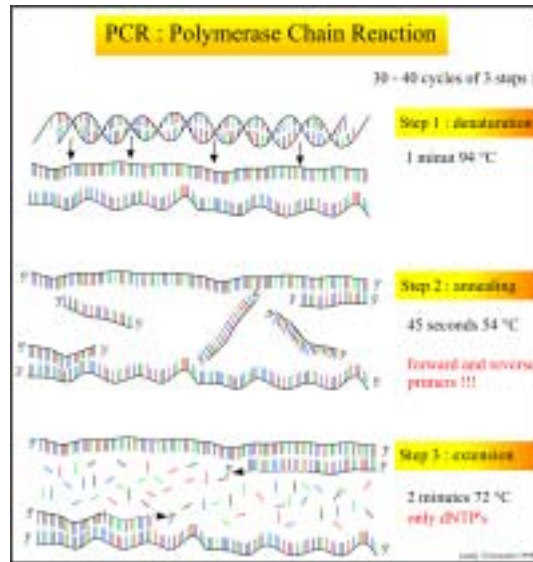


그림 5 PCR의 세부단계

2.2.5. 겔 전기 영동법

DNA의 길이에 따라 분자량이 다르고, 약간의 전기적 성질을 띠고 있기 때문에 양쪽에서 서로 다른 극의 전압을 걸어줌으로서 이동속도의 차이를 이용해서 각 DNA분자들을 분자량에 따라서 분리할 수 있다. 이를 통해서 원하는 길이의 DNA분자만을 골라내는 것이 가능하다.

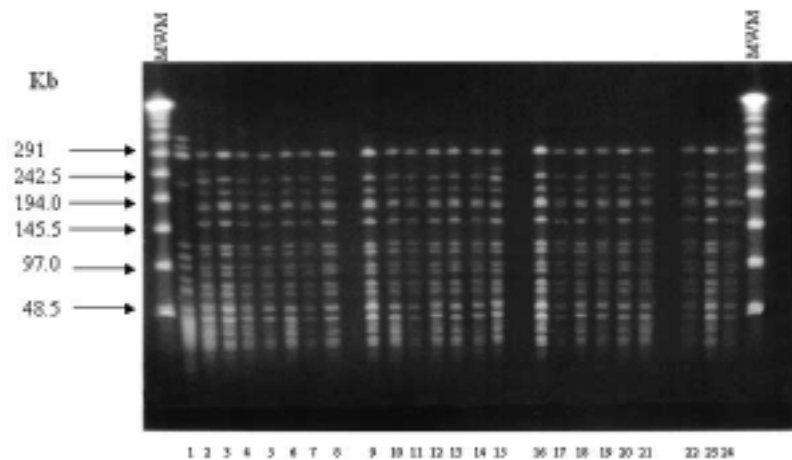


그림 6 겔 전기 영동법

2.3. DNA 컴퓨팅의 특징

DNA 컴퓨팅은 기존의 실리콘 기반의 컴퓨터가 할 수 없는 초병렬적인 연산을 해낼 수 있다. 이를 통해서 NP-complete문제와 같은 어렵다고 여겨지는 문제들을 짧은 시간 안에 해결할 수 있는 가능성을 가지고 있다. 하지만 이에 반해서 현재의 기술수준으로는 앞서 소개한 실험과정을 실행하기 위해서 많은 부가적인 시간과 함께 비용면에서도 부담이 큰 것이 문제이다.

또한 DNA 컴퓨팅은 정확한 연산이 어려우며 언제나 일정수준 이상의 오류 가능성을 내포하고 있다. 그러나 이 오류 중에서 몇 가지는 새로운 문제 해결 알고리즘의 기초가 될 수 있는데 본 논문의 실험에서는 상보결합이 잘못 일어나는 것을 이용한다. 즉 A와 T가 결합하고 C와 G가 결합하지 않고 A-C, A-G, T-C, T-G등의 결합이 일어나는 것을 이용하여 알고리즘이 구성되었다.

그러나 일반적인 경우 DNA 컴퓨팅의 발생 가능한 오류들은 여전히 해결이 필요한 문제로서 많은 연구가 계속되고 있다.

3. word game

3.1. word game의 모델

우리가 다룰 핵심적인 구조는 주어진 ‘단어’가 있을 때 이것과 연관성 깊은 정해진 개수의 다른 단어를 출력해내는 시스템이다. 이는 인간의 두뇌내부에서 일어나는 언어처리의 일부분을 모델링 하는 것이다. 이는 아직 뇌과학의 영역에서도 어떠한 메커니즘을 가지고 이루어지는지는 정확히 알려지지 않고 있다.

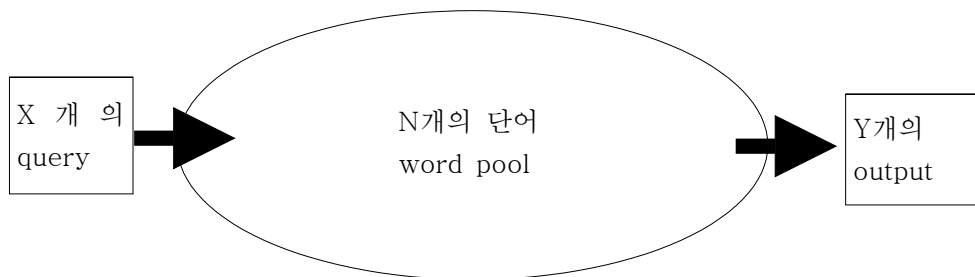


그림 7 다루는 문제의 개략도.

이 문제를 좀더 정량적으로 다시 표현하자면 N개의 주어진 단어들 중에서 어떤 주어진 X개의 단어들이 있을 때 이 X개의 단어와 동시에 관련이 있는 Y개의 단어를 찾아내서 출력하는 시스템이다.

이때 관련성을 하나의 변수로 준다면 주어진 N개의 단어에 순서대로 번호를 매기고, i번째 단어와 j번째 단어 사이의 연관도를 R_{ij} 하자. R_{ij} 는 0과 1의 사이를 갖고, 0은 두 단어가 전혀 관련 없다는 것을 의미하고 1은 두 단어가 완벽한 관련을 맺고 있다는 것을 뜻한다.

표 1 순차적이고 직렬적인 처리를 위한 테이블의 예

i		R_{i1}	R_{i2}	...
1	Ada	X	0.1	...
2	Art	0.1	X	...
3	Bit	0.5	0.01	...
		...	...	...

3.2. 기존 시스템의 한계

기존의 순차적이고 직렬적인 처리를 이용한 시스템에서는 word game을 처리함에 있어서 인간의 지능 수준의 처리를 바라보기는 커녕 기하급수적으로 늘어나는 기억용량과, 처리연산 그로 인한 시간 복잡도 등의 문제가 매우 크다. 이 문제에 대한 기존의 접근 방법은 $N \times N$ 개의 기억장소를 가진 테이블

에 각 단어간의 연관도를 저장해서 입력으로 들어온 X개의 단어에 해당되는 연관도를 곱해서 가장 큰 값을 지니는 것부터 Y개를 선택해서 출력하는 방식이다.

이러한 방법으로 N개의 단어에서 X개의 입력단어에 관련 있는 Y개의 단어를 찾아내기 위해서는 $O(N^2)$ 의 기억장소가 필요하고 $O(NX)$ 의 수행시간이 필요하게 된다.

표 2 N의 크기에 따른 필요한 기억용량의 예

N	필요한 기억용량(4byte/each Rij)
10^2	40Kilobyte
10^3	4Megabyte
10^4	400Megabyte
10^5	40Gigabyte

영어의 경우 일반인이 약 10^5 개의 단어를 사용한다고 알려져 있으므로 기존 방식으로 접근하기 위해서는 10^{10} 개의 매우 거대한 기억장소가 필요하고 수행 시간 역시 매우 크다.

3.3. 새로운 접근방법

이러한 점에서 우리의 두뇌 속에서 이루어지는 word game같은 언어 처리 과정은 아직 뇌과학의 영역에서도 거의 밝혀진 것이 없는 상황이지만 최소한 직렬적이고 순차적인 방식이 아님은 분명하다. 이러한 측면에서 우리는 완벽하지는 않지만 조금이나마 인간의 뇌가 처리하는 방식을 DNA를 연산의 도구로 사용하여, 초병렬적인 대량 계산을 이용하여 모델링 할 수 있다. 이러한 접근 방법은 데이터의 양이 늘어날수록 기존의 방식보다 더 효과적인 정보처리 능력을 보여줄 수 있다.

다음 장에서 다시 다루겠지만 본 논문은 이러한 기존 시스템과의 비교를 위하여 word game을 DNA컴퓨팅을 이용하여 접근하여, 다음과 같은 기본적 디자인을 설계하고 실행하였다. 주어진 N개의 단어들을 DNA코드로 변환하여 섞어놓은 후 입력query를 넣고 혼합한다. 일정한 시간 뒤에 입력단어와 결합한 단어들의 코드를 추출하여 어떤 단어인지를 decode하여 결과값으로 출력하는 방식이다. 이것은 단어의 개수 N이 증가해도 상수 시간의 시간복잡도를 가진다.

물론 DNA 컴퓨팅 자체가 가지는 문제점으로 인한 오버헤드와, 기술적 어려

움이 존재한다. 실질적인 실험 과정과, 파생되는 문제점 및 해결방안 등은 다음 장에서 논하기로 한다.

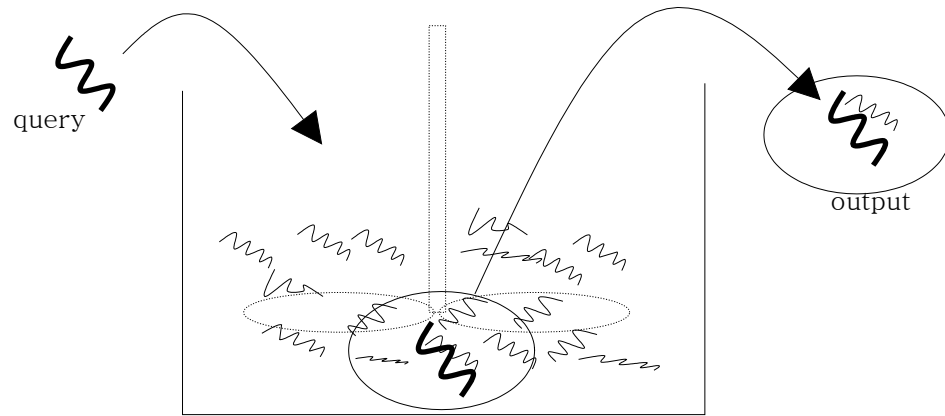


그림 8 DNA 컴퓨팅을 이용한 word game 개략도

4. 핵심 디자인

4.1. 문제점 및 해결 전략

word game의 핵심은 연관성이 있는 단어를 얼마나 빨리 골라낼 수 있는가 하는데 달려있다. 본 연구에서는 DNA의 염기의 상보적인 수소결합을 이용한다. 즉 두 단어가 연관성이 높을수록 DNA코드가 비슷한 염기서열을 갖도록 코딩하여서 query로 주어진 상보적인 배열과 쉽게 결합하고 또 연관성이 높을수록 결합력이 큰 것을 이용하여 최대한 비슷한 시퀀스를 골라내고, 이를 디코딩하여 연관성이 큰 단어들을 output으로 출력해낼 수 있도록 한다.

이러한 프로세싱을 거치는 word game을 DNA 컴퓨팅으로 모델링하기 위해서는 다음과 같은 요소들을 감안하여 실험과정과, 코드가 디자인되어야 한다.

4.1.1. 각 워드의 DNA 코드 디자인

이 문제의 선행조건은 각각의 단어의 관련성을 코드에 어떻게 반영시키는 것인가 하는 것이다. 예를 들어 관련성이 1에 가까운 두 단어는 DNA의 코드에 있어 염기 배열이 적게 차이 나도록 코딩하는 것이 직관적이나, 이럴 경우 나머지의 수많은 다른 단어들을 역시 이러한 기준으로 맞추어야하므로 기존 방식의 접근 방법만큼이나 큰 기억공간과, 수행시간을 요구할 수 있다. 따라서 N의 개수가 늘어날수록 이러한 코드 작성의 오버헤드를 예상하고 오버헤드를 줄이는 방법을 생각해야 했다. 그러나 본 논문에서의 핵심은 각 단어를 코딩하는데 있어 실질적인 연관성을 얼마나 적용할 수 있는지의 문제가 아니라 이러한 데이터 즉, word pool이 주어졌을 경우 DNA코드로 어떻게 코딩하고, 또한 어떠한 실험과정을 거쳐서 원하는 결과를 얻어낼 수 있는가에 주안점을 두고 있다. 따라서 본 연구에서는 이러한 문제를 해결하기 위해 word들을 모두 같은 길이의 시퀀스로 코딩하며, 각 시퀀스는 다수의 특성 필드를 지니게 하여, 각 필드마다 그 필드의 주제와의 연관성을 High, Medium, Low의 세가지 단계로 대략적으로 가정하여 코딩하였고, 마지막 필드는 그전 특성 필드가 동일하게 코딩된 경우 전체 시퀀스를 구별하기 위하여 추가되어 구별자의 역할을 한다.

4.1.2. query 디자인

가장 쉬운 방법은 단어들이 DNA코드로 변환된 서열을 G-C A-T의 상보적 염기서열로 바꾸어서 입력단어의 형식으로 삼는 것이다. 즉 간단하게 축약한 예를 들면 어떤 임의의 단어 X가 AAGG로 변환되어 word pool에 들어있다면 입력단어로 X가 있는 경우 query는 TTCC로 코딩할 수 있다. 그러나 이 경우

기존 word pool의 X와 입력단어로 투입된 X가 상보적으로 일치함으로 결합되어버림으로서 정작 필요한 다른 연관단어를 찾지 못할 가능성이 있다. 이는 실험의 마지막 단계인 output출력 과정에서 output이 X자체인지를 확인하여 제거하는 단계를 추가함으로서 해결할 수 있다.

또한 query의 길이가 길면 각 필드에 정확히 대응되지 않고, 필드의 경계를 넘나들면서 hybridization의 잘못된 오류가 발생할 수 있다. 따라서 각 필드별로 High, Medium, Low의 값을 다른 코드로 작성하고 query로 투입할 때는 각 필드별로 잘라진 단어를 투입하도록 설계하였다. 이를 통하면 많은 query와 hybridization된 word를 겹 전기영동법을 통해 추출해내면 연관성이 더 높은 단어를 자동적으로 소팅하여 출력할 수 있는 효과도 기대할 수 있고, 또한 긴 query를 이용할 때 발생하는 일부의 sequence에서만 hybridization이 일어나서 의도하지 않은 output이 발생하는 것을 막을 수 있다. 즉 여러개의 query DNA 시퀀스가 결합해서 분자량이 커질 수록 우리가 원하는 output일 확률이 높아진다.

복수의 단어가 query로 주어질 경우에는 각각의 단어를 query로 변환해서 집어넣어서 처리하면 된다. 그러나 실질적인 실험을 해보면 query를 이루는 단어가 많아지면 많아질수록 hybridization이 일어나는 빈도가 평균에 가까워져서 output을 추출하기가 어려워지는 문제점이 발생하는데 이는 다음장에서 더욱 자세히 다루도록 한다.

4.1.3. output의 문제

-Y개의 원하는 출력값보다 많은 단어가 추출되었으면 이 중 입력단어와 더 관련이 깊은 단어를 반환해야하는데 여기에도 새로운 비용이 든다. 따라서 이미 추출된 단어 중에서도 입력단어와 어떤 것이 더 연관도가 높은지 비교할 필요성이 있다. 앞서 밝힌대로, query가 각 필드별로 분리되어 투입됨으로서 각 query 분자가 많이 결합한 word가 더 적합한 output로 인정하고 출력할 수 있다.

4.1.4. 제한 및 가정

앞에서 다룬 문제점 외에도 각 DNA분자의 적절한 길이와, DNA 코드의 개수, 그리고 실험을 다루는 실제 온도 등의 다양한 실험조건이 실제 DNA분자를 가지고 하는 실험에서는 파생되고 어떻게 설정하느냐에 따라 실험 결과에 큰 영향을 미칠 수 있다. 하지만 본 논문에서는 이러한 제한 조건들에 얽매이지 않고 최선의 조건을 가정한 후, 이론적이지만 핵심적 디자인을 차용한 실험 알고리즘 및 DNA 코딩을 적용하여 시뮬레이션 하여 DNA 컴퓨팅의 가능성을 시험하는데 전력하기로 한다. 또한 여기서 발생하는 다양한 가정들 또한 새로

운 연구과제로서 큰 의미가 있을 것이다.

4.2. 실험 알고리즘

4.2.1. step1-query 생성

query를 생성한다. query는 입력단어의 상보적 배열이며, 입력단어가 복수일 경우는 입력단어 각각에 대하여 상보적 배열을 만든다. 이때 각 필드별로 나누어 필드 개수만큼 생성한다.



그림 9 query생성-각각의 색깔은 각 필드를 표시하고 아래의 화살표가 word pool내의 DNA코드, 위의 네 개의 화살표가 생성한 query를 개략적으로 표시해준다.(앞으로의 개략도는 각 단어의 DNA 코드가 4개의 필드로 이루어져 있는 것을 가정한 것이다)

4.2.2. step2-query 투입

step1에서 생성한 query를 word pool에 투입하여 merging시킨다. 이를 통해 다음 단계인 hybridization이 이루어질 수 있다.

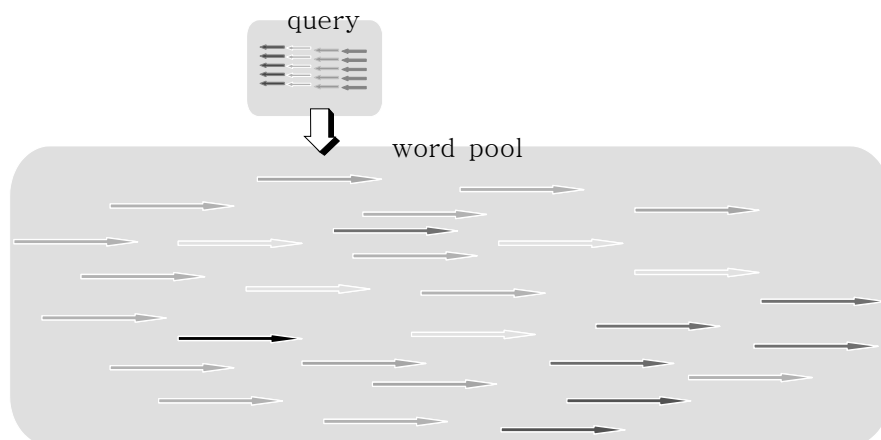


그림 10 query투입-색깔이 다른 것은 서로 다른 시퀀스임을 의미한다

4.2.3. step3-hybridization

query sequence와 word pool내의 word sequence간에 hybridization 반응을 일으킨다. 이때 query와 word간의 연관도에 따라서 다양한 크기의 dsDNA를 생성한다. 만약 query와 전혀 일치하지 않는 word라면 이 단계가 끝난후에도 ssDNA로 남아있을 것이고 완벽히 일치한다면 완벽한 dsDNA가 될 것이다. 단 이 실험 단계에서는 적절한 온도의 조절이 필요하다. 즉 hybridization이 거의 모든 입력 query에 대해서 일어날 수 있을 정도의 온도가 필요하다. 이때

같은 word에 붙은 query들끼리는 ligation이 일어나서는 안된다. 만약 ligation이 일어나면 실험결과에 커다란 오류를 일으킬 수 있다.

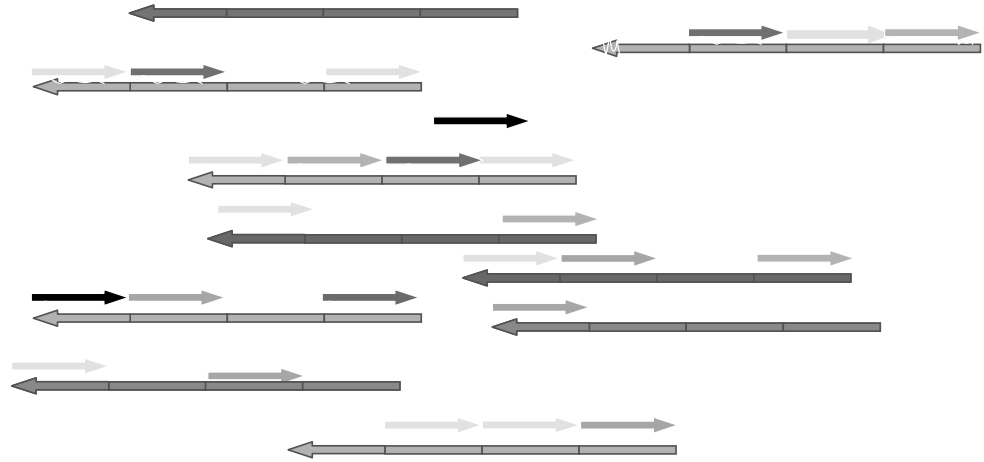


그림 11 word와 query의 hybridization

4.2.4. step4-gel electrophoresis

step3의 결과물에서 겔 전기 영동법을 이용하여 hybridization이 일어난 DNA분자들만을 추출해 낸다. 이 DNA분자들은 일부가 dsDNA형태로 되어있다. 이때 hybridization이 일어나지 않은 분자들중 query로 투입된 분자들은 이때 분자량이 원래 word크기보다 훨씬 작으므로 제거하고 나머지 word 시퀀스는 word pool로 되돌려보낸다. 이때 각 필드의 query가 몇 개나 결합했느냐에 따라 dsDNA의 분자량이 일정한 임계값에 따라 구분되고 제각기 추출될 수 있다.

또 다른 문제로서 DNA의 분자량이 커질수록 ssDNA 자체가 헤어핀 구조를 이루는 등의 결합오류가 커질 수 있지만, 본 실험 과정을 통하면 헤어핀 구조가 이루어진다고 해도 분자량은 증가하지 않기 때문에 문제를 일으키지 않는다.



그림 12 겔 전기영동법을 통한 dsDNA분리

4.2.5. step5-melting

step4의 결과로 추출된 dsDNA를 ssDNA로 만든다. 이 단계 내에서 온도조절과 겔 영동법을 추가한다면 여러 개의 염기서열 중 일부만 결합한 잘못된 오류들을 제거할 수 있다. 이는 여러 염기가 결합한 경우가 온도가 높아져도 결합을 유지한다는 사실에 기초하고 있다.

4.2.6. step6-gel electrophoresis

step5의 결과물에서 query인 DNA분자들을 제거하기 위해서 겔 전기 영동법을 사용하여 word의 크기의 분자량의 DNA만을 추출하고 나머지 query문은 제거한다.

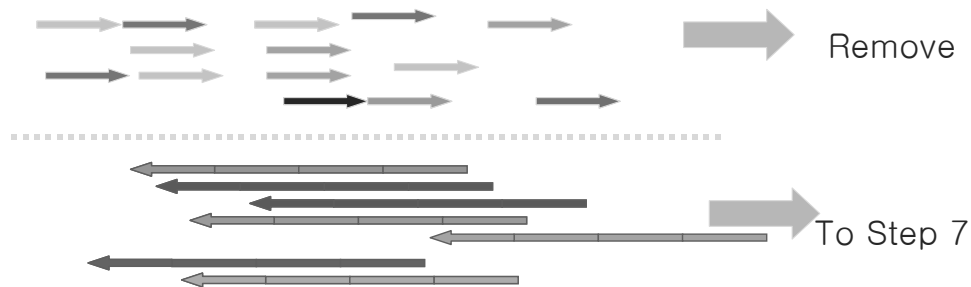


그림 13 겔 전기 영동법을 이용한 query와 word의 분리

4.2.7. step7-sampling

step6의 결과물에서 DNA분자를 추출해서 decode하여 출력한다. 이때 step4에서 분자량에 따라 추출빈도를 달리함으로써 연관성이 큰 단어가 추출될 확률을 크게 한다. 또한 이 단어가 query를 구성하게 했던 입력단어 그 자체인 경우 출력을 취소하고 다른 분자를 찾아 출력한다.

4.2.8. step8-restore and goto step1

word pool을 원상태로 만들고 다른 입력단어를 받아들여 step1으로 되돌아간다. 즉 지금까지 word pool에서 추출했던 DNA 시퀀스는 모두 되돌리고, query는 제거하여 새로운 query를 받아들이는 동작을 계속 할 수 있다. 지금까지의 실험 과정 속에는 원래 word pool에 있는 DNA 시퀀스를 조작하는 과정이 hybridization밖에 없기 때문에 원상태로 되돌리는 과정이 가능하다.

5. 시뮬레이션 및 결과

5.1. 실험내용

앞서 설계한 실험과정을 실제로 적용해보기 위하여 컴퓨터 공학 분야의 단어 100개를 대상으로 하여 DNA코드를 디자인 하였다. 각 단어들의 특성은 DB, NETWORK, PROGRAMING, ARCHITECTURE, OS, SOFTWARE의 컴퓨터공학의 6가지 토픽과 함께 NOTE필드까지 총 일곱 개의 필드로 구성되며, 각 필드당 5mer씩 총 35mer의 크기를 가지고 각 필드의 주제에 관한 연관도를 High, Medium, Low로 가정하여 디자인 하였다.

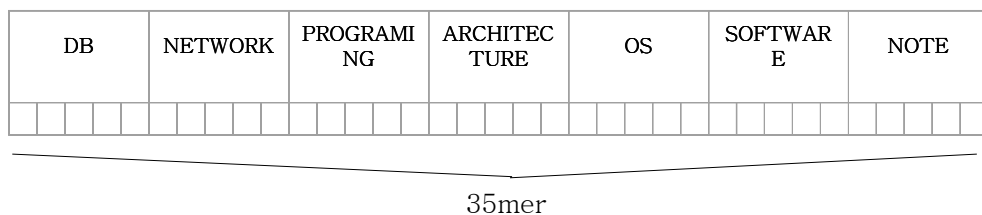


그림 14 실험 word의 DNA 코드 구조

비슷한 시퀀스가 반복되어 오류가 일어나는 것을 막기 위하여 각 필드는 서로 다른 시퀀스로 디자인 되었고 각 필드별 특성값에 따른 코드 디자인은 <표 3>과 같다.

표 3 각 필드에 대한 특성값 지정

	DB	NETWORK	PROGRAMING	ARCHITECTURE	OS	SOFTWARE	NOTE
High	AGAAA	CACCC	GCGTA	AGCCG	TAGAT	GATGA	CATAG
Medium	AGAAT	CACCG	GCGTT	AGCTG	TGGAT	GACGA	AATAG ATTAG
Low	AGATT	CACGG	GCGAT	AGTTG	TGAAT	GACCA	CTTAG

코딩된 전체 단어의 목록은 <표 5>에 주어졌다. 이렇게 인코딩된 데이터를 이용하여 실제로 어떠한 값을 output으로 도출할 수 있는지 시뮬레이션하였다.

표 4 실험에 사용된 단어의 DNA 코드

address	AGAAACACCCGCGTTATCCGTAGATGACGACATAG
ASCII	AGATTCACCGGCGTAAGCTGTGAATGATGACATAG
ASP	AGATTCACCCGCGTAAGTTGTGAATGATGACTTAG
assembler	AGATTCACGGGCGTAAGCTGTGGATGACGACATAG
broadcast	AGATTCACCCGCGATAGTTGTGAATGATGACATAG
browser	AGAATCACCCGCGATAGTTGTGAATGATGACATAG

bus	AGATTCACGGGCGTTATCCGTGGATGACCAAATAG
byte	AGATTCACGGGCGTTATCCGTGGATGACGACTTAG
C++	AGAATCACGGGCGTAAGTTGTGGATGATGACATAG
cache	AGATTCACGGGCGTTATCCGTAGATGACCACATAG
class	AGAATCACGGGCGTAAGTTGTGAATGATGACATAG
compile	AGAATCACGGGCGTAAGCTGTGGATGACGACATAG
cookie	AGAAACACCCGCGATAGTTGTGAATGACGACATAG
database	AGAAACACCCGCGTTAGTTGTGAATGACGACATAG
deadlock	AGATTCACGGGCGATAGTTGTAGATGACGACATAG
Debug	AGAATCACGGGCGTAAGCTGTGAATGACGACATAG
download	AGATTCACCCGCGATAGTTGTGAATGACGACTTAG
duplex	AGATTCACCCGCGATAGCTGTGAATGACCACATAG
entity	AGAAACACGGGCGATAGTTGTGAATGACGACATAG
ethernet	AGATTCACCCGCGATAGTTGTGAATGACGACTTAG
FIFO	AGATTCACGGGCGTTAGCTGTGAATGATGACATAG
filtering	AGAATCACCCGCGTTAGTTGTGGATGATGACTTAG
firewall	AGATTCACCCGCGTTAGCTGTGGATGACGAAATAG
gateway	AGATTCACGGGCGTTAGTTGTGAATGACCACATAG
GNU	AGATTCACGGGCGTAAGTTGTAGATGATGACATAG
IEEE	AGAATCACGGGCGTTAGCTGTGGATGACGACATAG
Interrupt	AGATTCACGGGCGTTAGCTGTAGATGACGACATAG
IP	AGATTCACCCGCGATAGTTGTGAATGACCACATAG
Java	AGAATCACGGGCGTAAGTTGTGAATGATGACATAG
JDBC	AGAAACACGGGCGTAAGTTGTGAATGATGAAATAG
JScript	AGAATCACGGGCGTAAGTTGTGAATGATGACTTAG
Jump	AGATTCACGGGCGTTATCCGTGGATGACCAATTAG
layer	AGATTCACCCGCGATAGCTGTGAATGACGACATAG
LIFO	AGATTCACGGGCGTTAGCTGTAGATGATGACATAG
link	AGATTCACCCGCGTTAGCTGTGAATGACGAAATAG
lock	AGATTCACGGGCGTTAGTTGTAGATGACCACCTTAG
log	AGAATCACGGGCGTTAGTTGTGGATGACGACATAG
Macro	AGATTCACGGGCGTAAGCTGTGAATGACGACATAG
mailbox	AGATTCACGGGCGTTAGTTGTAGATGACGACATAG
Memory	AGATTCACGGGCGTAATCCGTAGATGACCACATAG
MFC	AGAATCACGGGCGTAAGTTGTGAATGACGAAATAG
middleware	AGAATCACGGGCGTTAGCTGTAGATGACGAAATAG
modeling	AGAAACACCCGCGTTATCCGTAGATGACGACATAG
MODEM	AGATTCACCCGCGATAGTTGTGAATGACCACATAG
MPEG	AGATTCACGGGCGTAAGTTGTGAATGACGACATAG
MySQL	AGAAACACCCGCGTTAGTTGTGAATGATGACATAG
network	AGATTCACCCGCGTTAGCTGTGAATGACGACATAG
NTFS	AGATTCACGGGCGATAGCTGTAGATGATGACATAG
object	AGAATCACCCGCGTAAGTTGTGAATGATGACATAG
Operand	AGATTCACGGGCGTAAGTTGTAGATGATGACATAG
Operation	AGAATCACGGGCGTAATCCGTAGATGATGACATAG
Operator	AGATTCACGGGCGTAAGCTGTAGATGATGACATAG
OS	AGAATCACGGGCGTTAGCTGTAGATGACGACATAG
overflow	AGATTCACGGGCGTAATCCGTGAATGACCACATAG
overhead	AGAATCACGGGCGTTAGCTGTGGATGACCACATAG
Overriding	AGATTCACGGGCGTAAGTTGTGAATGATGAAATAG

packet	AGATTCACCCGCGATAGTTGTGAATGACGAATTAG
Page	AGAATCACGGGCGTAAGTTGTAGATGATGACATAG
parser	AGATTCACGGGCGTAAGTTGTGGATGATGACATAG
password	AGAATCACGGGCGATAGTTGTGAATGACGACATAG
patch	AGATTCACGGGCGTTATCCGTGAATGACCACATAG
Path	AGAATCACGGGCGTTAGTTGTGGATGACGACATAG
performance	AGAATCACGGGCGTAATCCGTAGATGACGACATAG
PHP	AGAATCACCCGCGTAAGTTGTGAATGATGAATTAG
Pipeline	AGATTCACGGGCGTTATCCGTGGATGACCACATAG
Platform	AGAATCACGGGCGTTATCCGTAGATGACCAAATAG
Polymorphism	AGATTCACGGGCGTAAGTTGTGAATGATGAATTAG
POP	AGATTCACCCGCGTTAGTTGTGAATGACGACTTAG
port	AGATTCACCCGCGATAGCTGTGGATGACGACATAG
Program	AGAAACACCCGCGTAAGCTGTAGATGATGACATAG
Programmer	AGAAACACCCGCGTAAGTTGTGGATGATGACATAG
protocol	AGATTCACCCGCGTTAGTTGTAGATGACGACATAG
query	AGAAACACGGGCGTTAGTTGTGAATGATGAAATAG
register	AGATTCACGGGCGTTATCCGTAGATGACCACATAG
Router	AGATTCACCCGCGATAGTTGTGAATGACCACATAG
Sampling	AGAATCACGGGCGATAGTTGTGAATGACCACATAG
segment	AGAAACACCCGCGTAAGCTGTAGATGACGAAATAG
serial	AGAATCACGGGCGATATCCGTGAATGACGACATAG
server	AGAATCACCCGCGTTATCCGTAGATGATGACATAG
Service	AGAATCACCCGCGTTATCCGTGGATGATGACATAG
shell	AGAATCACGGGCGTTAGTTGTAGATGATGACATAG
Simulation	AGATTCACGGGCGTAAGCTGTGGATGACGACATAG
Smalltalk	AGATTCACGGGCGTAAGTTGTGAATGATGACTTAG
SMTP	AGATTCACCCGCGATAGTTGTGAATGACGACATAG
socket	AGATTCACCCGCGATAGTTGTGGATGACGACATAG
Software	AGAATCACGGGCGTAAGCTGTGGATGATGACATAG
source	AGAATCACGGGCGTAAGTTGTGGATGACCACATAG
stack	AGATTCACGGGCGTTATCCGTGGATGACGACATAG
switch	AGATTCACCCGCGTTAGTTGTGAATGACCACATAG
syntax	AGAATCACGGGCGTAAGTTGTGAATGACCACATAG
TCP	AGATTCACCCGCGATAGTTGTGAATGACGAAATAG
Telnet	AGATTCACCCGCGATAGTTGTGGATGATGACATAG
Template	AGATTCACGGGCGTAAGTTGTGAATGATGACATAG
thread	AGATTCACGGGCGTTAGCTGTAGATGACGACATAG
Token	AGATTCACGGGCGTAAGCTGTGAATGACGACATAG
traffic	AGATTCACCCGCGATAGTTGTGGATGACCACATAG
transaction	AGAAACACGGGCGTTAGTTGTGGATGACCACATAG
variable	AGAAACACGGGCGTAAGTTGTGAATGACGAAATAG
virus	AGATTCACGGGCGTTAGTTGTGGATGACGACATAG
Winsoc	AGATTCACCCGCGTTAGTTGTGGATGACGACATAG

5.2. 실험방법

시뮬레이션을 위해서 C언어를 사용하여 word 데이터를 받아들이어서 random 한 화학반응을 일으키는 것을 프로그래밍화 하였다. 이때의 핵심 조건 값들에 대한 정의는 다음과 같다.

표 5 실험에 관련된 핵심 조건 가정

조건	값
hybridization과정에서 에러발생률	0.01
투입된 query의 개수(각 필드별)	1000000
query의 개수 및 실제 word	1(TCP), 2(Transection, JAVA)
실험횟수	10

시뮬레이션 상의 편의를 위해서 시퀀스를 각 필드별 특성값을 수로 바꾸어 처리하도록 하였으며 랜덤 함수를 써서, 에러발생을 컨트롤 하였다. 우선 모든 query가 word pool안의 모든 DNA분자와 랜덤하게 맞부딪히고 이때 해당되는 각 필드에만 접근 가능하다고 가정하였다. 이때 각 필드의 특성값에 따라 hybridization이 일어나게 되는데 이때 hybridization이 일어나는 경우를 에러 발생률을 통해 계산한다. hybridization이 일어나지 않은 경우는 다른 분자를 찾아서 hybridization이 일어날 때까지 반복하게 된다. 모든 query분자가 word pool내의 DNA분자에 결합하게 된 경우 지금까지의 hybridization이 일어난 경우를 종합하여 query와 결합한 개수를 도출하였다. 이 결합한 개수를 output을 추출할 때의 가중치로 삼아, 연관성이 클수록 output으로 선택되도록 한다. 그렇지만 결국 랜덤 추출이기 때문에 query가 hybridization된 빈도에 비례하여 output이 출력된다. 따라서 hybridization의 빈도를 시뮬레이션 하는 것이 핵심이 된다.

5.3. 실험결과

실질적인 word인 TCP와 (Tranjection, Java)를 query로 투입하여 hybridization의 빈도 측정한 결과는 <그림 15>와 같다.

<그림 15>의 그래프에서 볼 수 있듯이 query인 TCP에서 유독 hybridization이 일어나고 점진적으로 연관도가 떨어짐에 따라 완만한 곡선을 그리며 감소한다. 따라서 우리는 output을 출력할 때 query그 자체인 TCP의 DNA 시퀀스를 제외해야 함을 알 수 있다. 그래프에는 잘 들어나지 않았지만 차례대로 link, packet, SMTP, download, ethernet 등의 단어들의 높은 빈도수를 보였다. 이는 코드 디자인 할때부터 높은 연관도를 가지고 있는 단어들이라서 올바른 결과값을 보인다고 할 수 있다.

두개의 단어가(transaction과 java) query로 주어진 경우에도 Mysql, source, C++, database, programmer등의 두 단어와 모두 연관된 단어를 찾을 수 있었다. 다만 <그림 16>의 그래프를 보면 <그림 15>의 그래프에 비해 각 단어의 hybridization 빈도의 편차가 줄어들고, 이로인해 곡선이 완만해짐을 알 수 있다. query가 더 많은 수의 단어로 이루어져 있을수록 더욱 심해짐을 전체 hybridization에서 각 단어에서 이루어진 비율을 query의 크기에 따라 비교한 <그림 17>의 그래프에서 추론할 수 있다. 따라서 각 단어 별로 연관도의 이러

한 작은 차이를 구별해서 적절한 가중치를 주어야 하기 때문에, 따라서 query를 이루는 단어의 개수가 많아질수록 적절한 output을 추출하기 위해서는 더욱 정교한 sampling기술이 필요하다.

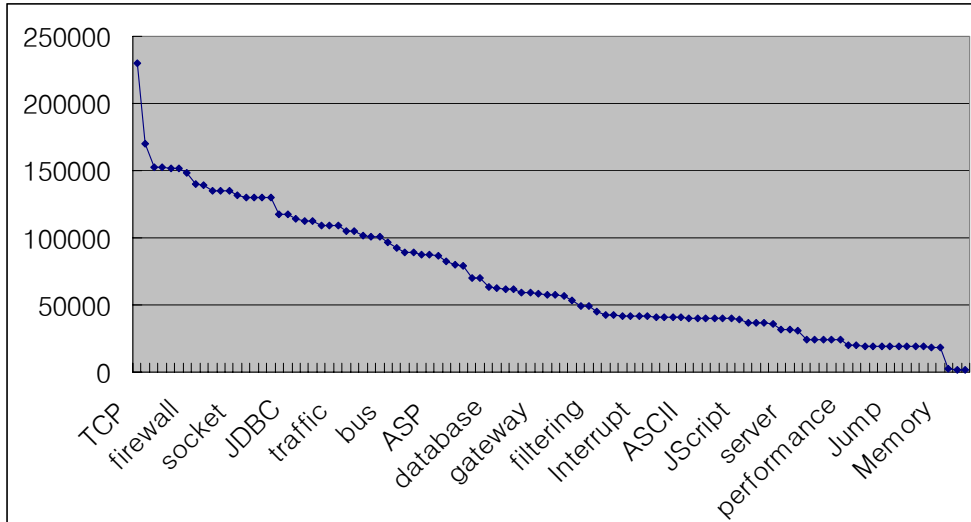


그림 15 TCP를 query로 투입했을 때 각 단어별 hybridization이 일어나는 빈도측정값의 평균 그래프(query의 개수 $N=7 \times 10,000,000$ 개, 실험횟수 10)

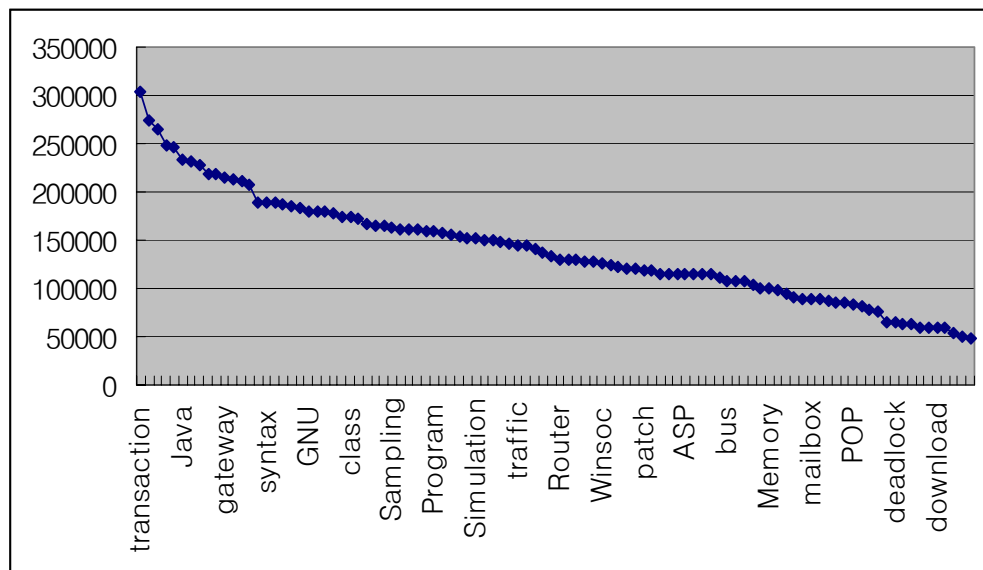


그림 16 transaction과 Java를 query로 투입했을 때 각 단어별 hybridization이 일어나는 빈도측정값의 평균 그래프(query의 개수 $N=2 \times 7 \times 10,000,000$ 개 실험횟수 10)

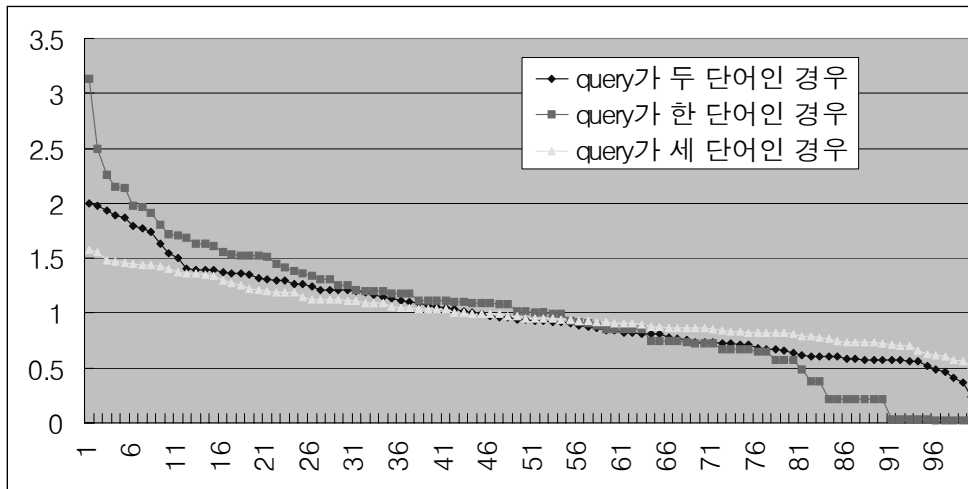


그림 17 query의 word수에 따른 각 단어별 hybridization의 비율
그래프(1~3개의 word query)

5.4. 결과분석

실험결과 simulation이기 때문에 실험과정상의 시간적 효율은 정확하게 측정할 수는 없었지만, 본 연구의 핵심이 초병렬성을 이용하면서 얼마나 기존의 방법과 비슷한 결과를 도출할 수 있느냐에 있기 때문에 hybridization에 걸리는 시간이 상수 시간임을 고려하면 결국 DNA 컴퓨팅을 통하여 기존의 순차적이고 직렬적인 접근과 비슷한 결과물을 얻어낼 수 있다는 것을 알 수 있었다. 완벽히 시뮬레이션 하지는 못했지만 기존의 방법이 같은 query에 대해서 늘 같은 결과물을 내놓는 것에 반하여 DNA 컴퓨팅으로 접근은 실험할 때마다 output이 달라지는 것을 알 수 있다. 특히 query가 여러 개의 단어로 이루어질 경우 각각의 단어에 query의 결합정도에 따른 가중치를 두기가 어려워지므로 좀 더 다양한 단어가 output으로 선택될 확률이 커지게 된다.

6. 결론

6.1. 연구 결과 요약

컴퓨터공학은 지금까지 실리콘기반의 컴퓨터를 가지고 매우 빠른 속도로 발전해왔다. 그러나 쉽게 풀 수 있는 문제에 대한 해결은 상상할 수 없을 정도로 빠르고 정확한 결과를 보여주고 있는 반면, 그렇지 않은 문제, 예를 들어 NP완비 문제 혹은 인공 지능 등에 관해서는 해결 방법이 매우 더디게 개발되고 있었고, 아예 해결방안을 못찾고 있기도 했다. 이럴 때 기존의 방법을 완전히 뒤집는 분자 컴퓨팅 기술, 그중에서도 DNA 컴퓨팅의 개념은 매우 충격적이고, 파급효과가 크다고 하겠다.

DNA 컴퓨팅은 아직 시작단계이고, 해결해야할 무수히 많은 문제점을 아직 안고 있다. 본 연구에서 다룬 문제도 아직까지는 기존의 컴퓨터를 이용하여 처리하는 것이 더욱 빠르고 간편한 것이 사실이고, 연구 과정 속에서 새로운 문제점들이 계속 파생되었다. 그럼에도 불구하고 본 연구에서는 순간적이고 병렬적으로 일어나는 인간의 두뇌 속의 언어처리 활동을 모델링 해봄으로서 기존의 방법론과는 다른 특징을 가지고 문제에 접근하여 해결할 수 있음을 확인하였다. 따라서 이는 DNA 컴퓨팅이 기존의 방법론으로 한계에 부딪힌 분야와 관련된 다양한 문제에 대한 새로운 접근의 한 예가 될 수 있음을 증명하는 것이다.

6.2. 연구 결과 활용 방안

실험 과정을 조금만 더 잘 다듬는다면 본 연구의 알고리즘은 다양한 분야에 응용될 수 있다. 가장 직접적으로 인간의 두뇌 속에서 이루어지는 단어의 연상 작용을 인공적으로 구현하는데 도움이 될 수 있을 것이고, 더 나아가 인간의 학습, 추론 등에도 적용될 수 있다.

또한 각 특성마다 일치하는 것을 query로 켜서 가장 많이 일치하는 것을 추출해내는 방식은 데이터베이스 분야에도 적용이 가능할 것으로 보인다. 특히 요즘 각광받고 있는 데이터마이닝에 있어서도 다양한 특성들 속에서 원하는 데이터를 추출해내는 본 연구의 알고리즘이 잘 적용될 가능성이 매우 높다고 하겠다.

6.3. 향후 과제

이번 연구에서의 시뮬레이션은 매우 많은 제약 조건을 가정하고 실행되었다. 따라서 실제 실험을 통해 word game을 구현하기 위해서는 각 조건에 따른 화학반응의 변화에 대한 구체적인 데이터와, 실제적 비용을 줄일 수 있는 다양한 연구가 필요하다.

뿐만 아니라, 실험 프로토콜 자체도 query가 여러 단어일 때 나타나는 output의 비명확성 등의 새로운 문제를 여전히 안고 있기 때문에 이를 해결하

기 위해서 다양한 아이디어의 개발이 필요하다고 하겠다.

참고문헌

1. Adleman, L., Molecular computation of solutions to combinatorial problems. Science, 266:1021-1024, 1994.
2. Cormen, Thomas h., Leiserson, Charles E., and Rivest, Ronald L., Introduction to Algorithms., The MIT Press., 1994
3. Lee, I.-H., Park, J.-Y., Jang, H.-M., Chai, Y.-G., and Zhang, B.-T., DNA Implementation of Theorem Proving with Resolution Refutation in Propositional Logic, Lecture Notes in Computer Science, 2568:156-167, 2003.
4. Marder, E., Thirumalai, V., Cellular, synaptic and network effects of neuromodulation, Neural Networks, 15(4-6), 479-494, 2002.
5. Nilsson, N.J., Artificial Intelligence: A New Synthesis, Morgan Kaufman Publishers Inc., 1998.
6. Park, T.-H., Lee, J.-Y., and Zhang, B.-T., DNA Computing (in Korean), News and Information for Chemical Engineers, vol. 20: 553-563, 2002.
7. 김유진. 「단어연상에 있어서 연상의 계층적 강도의 몇가지 특성」. 석사학위논문. 서울대학교, 1969.
8. 신수용. 「DNA 컴퓨팅 기법을 이용한 조합 최적화 문제의 해결」. 석사학위논문. 서울대학교, 2000.
9. 장병탁. 「분자정보처리기술」, 『전자공학회지』 2002. 3: 286-298.
10. 정희재. 「분자컴퓨팅을 이용한 기억기반학습」. 석사학위논문. 서울대학교, 2003.
11. 이은석. 「DNAgram: Anagram 문제 해결에 관한 분자 컴퓨팅 시뮬레이션 연구」. 서울대학교, 2003.
12. http://elchem.kaist.ac.kr/BK21_SMS.web/2001_instanal/FIG/20011210/Univ_of_Ghent/principles/pcr.html
13. <http://phdcho.netian.com/cellmaterial.htm>
14. <http://www.cdc.gov/ncidod/eid/vol8no9/01-0435-G.htm>

부록

simulation program source code

datafile